Reviewer Pack — Betti Number Exponential Lower Bound on Tseitin Resolution L...

Entry 9c6ddd093b59 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 17:36:02 UTC

Betti Number Exponential Lower Bound on Tseitin Resolution Length

Entry ID: `9c6ddd093b59` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 17:36:02 UTC

1. Conjecture

Field A (mathematical branch): Algebraic topology (Betti numbers of clique complexes) **Field B** (complexity object): Resolution length of Tseitin formulas

Statement:

> For a graph G, the resolution length of its Tseitin formula is at least 2^{β₁(G)}, where β₁(G) is the first Betti number of the clique complex of G. This holds for all graphs with β₁(G) ≥ 1 and n ≤ 40.

Rationale (proposer's reasoning):

> Higher Betti numbers indicate nontrivial topological structure in the clique complex, which may necessitate exponentially longer resolution proofs due to the increased combinatorial complexity of the Tseitin formula's dependencies.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `246d5a2883073efc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_random_graph(n: int) -> list:
    graph = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                graph[i][j] = 1
                graph[j][i] = 1
    return graph

def simplicial_homology(graph: list) -> int:
    n = len(graph)
    beta_1 = 0

    # Find all edges (1-simplices)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n) if graph[i][j] == 1]

    # Find all triangles (2-simplices)
    triangles = []
    for i, j, k in combinations(range(n), 3):
        if graph[i][j] == 1 and graph[j][k] == 1 and graph[k][i] == 1:
            triangles.append((i, j, k))

    # Calculate beta_1
    beta_1 = len(edges) - len(triangles)

    return beta_1

def resolution_length(graph: list) -> int:
    n = len(graph)
    clauses = []
    for i in range(n):
        clause = [j + 1 if graph[i][j] == 0 else -(j + 1) for j in range(n)]
        clauses.append(clause)

    length = 0
    while clauses:
```

```
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if not unit_clause:
            break
        literal = abs(unit_clause[0])
        clauses = [c for c in clauses if literal not in c and -literal not in c]
        length += 1

    return length

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    graph = generate_random_graph(n)

    beta_1 = simplicial_homology(graph)
    if beta_1 < 1:
        return {
            "metric_name": "resolution_length",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "beta_1(G) < 1"
        }

    resolution_len = resolution_length(graph)
    if resolution_len < 2 ** beta_1:
        return {
            "metric_name": "resolution_length",
            "metric_value": resolution_len,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Resolution length {resolution_len} is less than 2^{beta_1}"
        }

    return {
        "metric_name": "resolution_length",
        "metric_value": resolution_len,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ecture_holds': False, 'counterexample': 'beta_1(G) < 1'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'resolution_length', 'metric_value': None, 'instances_tested': 1, 'conjecture
RESULT: FALSIFIED counterexample="beta_1(G) < 1" first_failing_seed=0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> { “critic_verdict”: “CHALLENGE”, “reasoning”: “The conjecture explicitly requires $\beta_1(G)$

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	89542
2	preregistration	ollama_remote	qwen3:8b	0	0	24174
3	novelty	ollama_remote	qwen3:8b	0	0	20792
4	novelty	ollama_remote	qwen3:8b	0	0	14891
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14589
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11042
7	critic	ollama_remote	qwen3:8b	0	0	30066

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	judge	ollama_remote	qwen3:8b	0	0	23842

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 228937 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/9c6ddd093b59.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9c6ddd093b59.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9c6ddd093b59.tar.gz` (if generated)